

Unit 4.4: Regression and Classification with Linear Models

Machine Learning Fundamentals

Artificial Intelligence Lecture
by
Kiran Bagale

IOE, Thapathali Campus

Jan, 2026

Outline

- 1 Introduction to Linear Models
- 2 Linear Regression
- 3 Logistic Regression
- 4 Practical Considerations
- 5 Summary and Extensions

Learning Objectives

By the end of this lecture, you will be able to:

- Understand the mathematical foundations of linear models
- Implement linear regression for continuous prediction tasks
- Apply logistic regression for binary classification
- Derive and implement gradient descent optimization
- Evaluate model performance using appropriate metrics
- Recognize when linear models are appropriate

What are Linear Models?

Definition

Linear models predict outputs as a **linear combination** of input features:

$$f(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d = w_0 + \mathbf{w}^T \mathbf{x}$$

Key Components:

- $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$: Input feature vector
- $\mathbf{w} = [w_1, w_2, \dots, w_d]^T$: Weight vector
- w_0 : Bias term (intercept)
- d : Number of features

Notation: Often we absorb bias into $\mathbf{w}$ by adding $x_0 = 1$

Why Linear Models?

Advantages:

- + Simple and interpretable
- + Fast to train and predict
- + Low computational cost
- + Well-understood theory
- + Work well with many features
- + Baseline for comparison

Limitations:

- Limited to linear relationships
- Poor for complex patterns
- Sensitive to outliers
- Feature engineering needed

Note: Can be extended with non-linear feature transformations!

Linear Regression: Problem Formulation

Objective

Given training data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, find weights $\mathbf{w}$ that best predict continuous target y .

Model:

$$\hat{y} = h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = \sum_{j=0}^d w_j x_j$$

Examples:

- Predicting house prices from features (size, location, age)
- Estimating temperature from sensor readings
- Forecasting stock prices from historical data

Loss Function: Mean Squared Error

How do we measure prediction quality?

Mean Squared Error (MSE)

$$J(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})^2 = \frac{1}{2n} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

Why MSE?

- Penalizes large errors more than small ones
- Differentiable (smooth optimization)
- Convex function (unique global minimum)
- Factor of $\frac{1}{2}$ simplifies derivatives

Matrix Formulation

Compact notation using matrices:

Design matrix $\mathbf{X}$:

$$\mathbf{X} = \begin{bmatrix} (\mathbf{x}^{(1)})^T \\ (\mathbf{x}^{(2)})^T \\ \vdots \\ (\mathbf{x}^{(n)})^T \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} & \cdots & x_d^{(n)} \end{bmatrix}_{n \times (d+1)}$$

Target vector: $\mathbf{y} = [y^{(1)}, y^{(2)}, \dots, y^{(n)}]^T$

MSE in Matrix Form

$$J(\mathbf{w}) = \frac{1}{2n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{2n} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Closed-Form Solution: Normal Equation

Minimizing MSE by setting gradient to zero:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{n} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y}) = \mathbf{0}$$

Normal Equation

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Properties:

- Direct analytical solution (no iteration needed)
- Requires $\mathbf{X}^T \mathbf{X}$ to be invertible
- Computational complexity: $O(d^3 + nd^2)$
- Impractical for large d or n ($d > 10,000$)

Gradient Descent for Linear Regression

Iterative optimization approach:

Gradient of MSE:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Vector form: $\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{n} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$

Gradient Descent Update Rule

$$\mathbf{w} := \mathbf{w} - \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

where $\alpha > 0$ is the learning rate

Gradient Descent Algorithm

Algorithm 1 Batch Gradient Descent for Linear Regression

- 1: **Input:** Training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, learning rate α , tolerance ϵ
 - 2: **Initialize:** $\mathbf{w} \leftarrow \mathbf{0}$ (or random small values)
 - 3: **repeat**
 - 4: $\mathbf{w}_{\text{old}} \leftarrow \mathbf{w}$
 - 5: Compute predictions: $\hat{\mathbf{y}} \leftarrow \mathbf{X}\mathbf{w}$
 - 6: Compute gradient: $\mathbf{g} \leftarrow \frac{1}{n}\mathbf{X}^T(\hat{\mathbf{y}} - \mathbf{y})$
 - 7: Update weights: $\mathbf{w} \leftarrow \mathbf{w} - \alpha\mathbf{g}$
 - 8: **until** $\|\mathbf{w} - \mathbf{w}_{\text{old}}\| < \epsilon$ or max iterations reached
 - 9: **Return:** $\mathbf{w}$
-

Time complexity per iteration: $O(nd)$

Stochastic Gradient Descent (SGD)

Problem with batch GD: Computing gradient over all n samples is expensive

SGD Update Rule

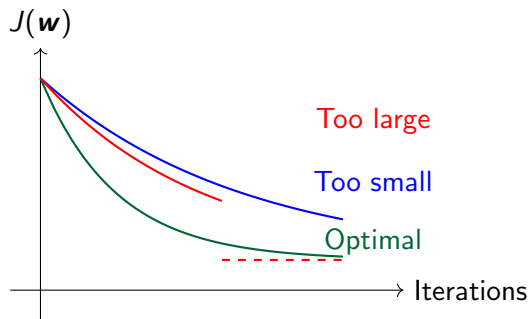
For each training example $(\mathbf{x}^{(i)}, y^{(i)})$:

$$\mathbf{w} := \mathbf{w} - \alpha(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})\mathbf{x}^{(i)}$$

Mini-batch Gradient Descent:

- Compromise between batch and stochastic
- Use subset (batch) of size b (e.g., 32, 64, 128)
- $\mathbf{w} := \mathbf{w} - \alpha \frac{1}{b} \sum_{i \in \text{batch}} (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)})\mathbf{x}^{(i)}$
- Faster convergence, better use of hardware parallelization

Learning Rate Selection



Guidelines:

- Start with $\alpha = 0.01$ or $\alpha = 0.1$
- Use learning rate schedules: $\alpha_t = \frac{\alpha_0}{1+kt}$, where k is the decay rate and t is the iteration or epoch
- Monitor loss: should decrease over iterations

Polynomial Regression

Extending linear regression to non-linear relationships:

Idea: Create polynomial features from original features

Example: Quadratic features from x :

$$\mathbf{x} = [1, x] \rightarrow \mathbf{x}_{\text{poly}} = [1, x, x^2]$$

General polynomial of degree p :

$$h_{\mathbf{w}}(x) = w_0 + w_1x + w_2x^2 + \dots + w_px^p$$

Multiple features: Include interactions

$$h_{\mathbf{w}}(x_1, x_2) = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2$$

Note: Still linear in parameters $\mathbf{w}$!

Regularization: Ridge Regression

Problem: Overfitting when many features or high polynomial degree

Ridge Regression (L2 Regularization)

$$J(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^d w_j^2$$

- $\lambda \geq 0$: Regularization parameter
- Penalizes large weights (prefers simpler models)
- Closed-form: $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$
- Always invertible when $\lambda > 0$

Lasso Regression (L1): $J(\mathbf{w}) = \text{MSE} + \lambda \sum_{j=1}^d |w_j|$

- Produces sparse solutions (feature selection)

Evaluation Metrics for Regression

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\text{MSE}} \quad (\text{same units as } y)$$

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}| \quad (\text{robust to outliers})$$

Coefficient of Determination (R^2):

$$R^2 = 1 - \frac{\sum_i (y^{(i)} - \hat{y}^{(i)})^2}{\sum_i (y^{(i)} - \bar{y})^2} \quad (\text{proportion of variance explained})$$

$R^2 = 1$: perfect fit, $R^2 = 0$: no better than mean

Classification vs Regression

Regression:

- Predict continuous values
- $y \in \mathbb{R}$
- Example: price, temperature
- Use linear model directly

Classification:

- Predict discrete classes
- $y \in \{0, 1, \dots, K - 1\}$
- Example: spam/not spam
- Need probability model

Why not use linear regression for classification?

- Linear regression outputs unbounded values
- We need $\hat{y} \in [0, 1]$ for binary classification
- Need to model probability: $P(y = 1|\mathbf{x})$

Logistic Regression: Motivation

Goal: Model $P(y = 1|\mathbf{x})$ for binary classification

Linear model problem: $\mathbf{w}^T \mathbf{x}$ can be any real number

Solution: Apply sigmoid function to constrain output to $[0, 1]$

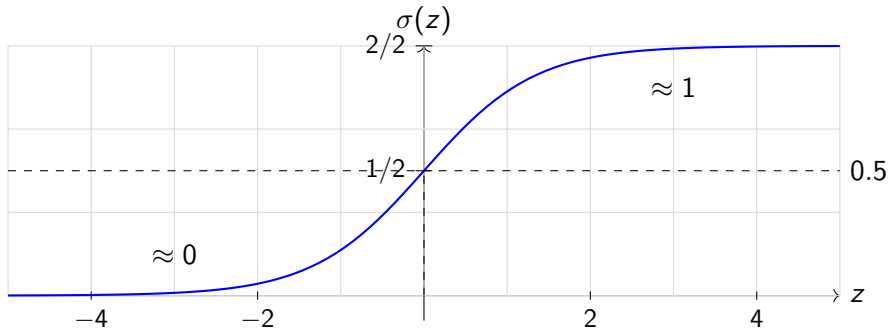
Sigmoid (Logistic) Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- $\sigma(z) \in (0, 1)$ for all $z \in \mathbb{R}$
- $\sigma(0) = 0.5$
- $\sigma(-z) = 1 - \sigma(z)$
- Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Sigmoid Function Visualization



Logistic Regression Model

Hypothesis Function

$$h_{\mathbf{w}}(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

Interpretation:

$$h_{\mathbf{w}}(\mathbf{x}) = P(y = 1 | \mathbf{x}; \mathbf{w})$$

Decision Boundary:

- Predict $y = 1$ if $h_{\mathbf{w}}(\mathbf{x}) \geq 0.5$ (i.e., $\mathbf{w}^T \mathbf{x} \geq 0$)
- Predict $y = 0$ if $h_{\mathbf{w}}(\mathbf{x}) < 0.5$ (i.e., $\mathbf{w}^T \mathbf{x} < 0$)
- Decision boundary: $\mathbf{w}^T \mathbf{x} = 0$ (linear in input space)

Loss Function: Cross-Entropy

Why not MSE? Not convex for sigmoid + leads to poor convergence

Binary Cross-Entropy (Log Loss)

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log(h_{\mathbf{w}}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right]$$

Intuition:

- If $y^{(i)} = 1$: loss is $-\log(h_{\mathbf{w}}(\mathbf{x}^{(i)}))$
 - Loss $\rightarrow 0$ as $h_{\mathbf{w}}(\mathbf{x}^{(i)}) \rightarrow 1$ (correct)
 - Loss $\rightarrow \infty$ as $h_{\mathbf{w}}(\mathbf{x}^{(i)}) \rightarrow 0$ (wrong)
- If $y^{(i)} = 0$: loss is $-\log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)}))$
- Derived from maximum likelihood estimation

Gradient of Cross-Entropy

Computing the gradient:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Remarkably: Same form as linear regression!

Gradient Descent for Logistic Regression

$$\mathbf{w} := \mathbf{w} - \frac{\alpha}{n} \sum_{i=1}^n (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)}$$

where $h_{\mathbf{w}}(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$

Note: No closed-form solution; must use iterative optimization

Regularized Logistic Regression

Prevent overfitting with regularization:

L2 Regularized Cross-Entropy

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log(h_{\mathbf{w}}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right] + \frac{\lambda}{2n} \sum_{j=1}^d w_j^2$$

Gradient with regularization:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{n} w_j$$

Note: Don't regularize bias term w_0

Multi-class Logistic Regression

Extension to K classes using softmax function:

Softmax Function

$$P(y = k | \mathbf{x}; \mathbf{W}) = \frac{e^{\mathbf{w}_k^T \mathbf{x}}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x}}}$$

where $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K]$

Properties:

- Outputs sum to 1: $\sum_{k=1}^K P(y = k | \mathbf{x}) = 1$
- Generalizes sigmoid (binary = $K = 2$ case)
- Each class has its own weight vector $\mathbf{w}_k$

Loss: Cross-entropy with one-hot encoded labels

Evaluation Metrics for Classification

Confusion Matrix for Binary Classification:

		Predicted	
		Positive	Negative
2*Actual	Positive	TP	FN
	Negative	FP	TN

Accuracy: $\frac{TP+TN}{TP+TN+FP+FN}$ (overall correctness)

Precision: $\frac{TP}{TP+FP}$ (of predicted positives, how many correct?)

Recall (Sensitivity): $\frac{TP}{TP+FN}$ (of actual positives, how many found?)

F1-Score: $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ (harmonic mean)

Specificity: $\frac{TN}{TN+FP}$ (true negative rate)

ROC Curve and AUC

Receiver Operating Characteristic (ROC) Curve:

- Plot True Positive Rate vs False Positive Rate
- Vary classification threshold from 0 to 1
- Shows trade-off between sensitivity and specificity

Area Under Curve (AUC):

- $AUC = 1$: Perfect classifier
- $AUC = 0.5$: Random classifier
- $AUC > 0.8$: Good classifier
- Threshold-independent metric

When to use:

- Class imbalance present
- Cost of FP $\neq$ cost of FN

Feature Scaling

Why scale features?

- Features on different scales cause slow convergence
- Large features dominate gradient
- Important for gradient descent (not for normal equation)

Min-Max Normalization:

$$x'_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)} \in [0, 1]$$

Standardization (Z-score):

$$x'_j = \frac{x_j - \mu_j}{\sigma_j} \quad (\text{mean } 0, \text{ std } 1)$$

Note: Apply same transformation to training and test data

Train-Test Split

Purpose: Evaluate model on unseen data to estimate generalization

Common splits:

- 80% training, 20% testing
- 70% training, 15% validation, 15% testing

k-Fold Cross-Validation:

- 1 Split data into k folds
- 2 Train on $k - 1$ folds, test on remaining fold
- 3 Repeat k times, rotating test fold
- 4 Average performance across folds

Benefits: Better use of limited data, more robust evaluation

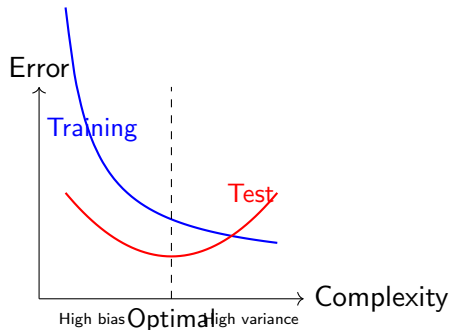
Bias-Variance Tradeoff

Bias (Underfitting):

- Model too simple
- High training error
- High test error
- **Solution:** Add features, increase complexity

Variance (Overfitting):

- Model too complex
- Low training error
- High test error
- **Solution:** Regularization, more data



Handling Non-linearity

Linear models can learn non-linear patterns through feature engineering:

1. Polynomial Features:

$$[x_1, x_2] \rightarrow [x_1, x_2, x_1^2, x_2^2, x_1 x_2]$$

2. Interaction Features:

$$[x_1, x_2, x_3] \rightarrow [x_1, x_2, x_3, x_1 x_2, x_1 x_3, x_2 x_3]$$

3. Non-linear Transformations:

$$[x] \rightarrow [x, \sqrt{x}, \log(x), e^x]$$

4. Domain-specific Features:

- Date/time $\rightarrow$ day of week, month, hour
- Text $\rightarrow$ word counts, TF-IDF

Caution: More features $\rightarrow$ higher risk of overfitting

Implementation Tips

Best Practices:

- 1 **Always** scale features before training
- 2 Monitor both training and validation loss
- 3 Start with simple model, increase complexity gradually
- 4 Use regularization to prevent overfitting
- 5 Check for convergence (loss plateaus)
- 6 Shuffle data before training
- 7 Use appropriate metrics for problem type
- 8 Visualize predictions vs actual values

Common Pitfalls:

- Data leakage (using test data in training)
- Not handling missing values
- Ignoring class imbalance
- Using accuracy for imbalanced datasets

When to Use Linear Models

Good for:

- Linear/nearly linear relationships
- High-dimensional sparse data
- Need for interpretability
- Baseline model
- Limited computational resources
- Small to medium datasets
- Online learning scenarios

Consider alternatives when:

- Highly non-linear patterns
- Complex feature interactions
- Lots of training data available
- Accuracy more important than speed
- Ensemble methods needed

Alternatives:

- Decision trees
- Neural networks
- Support vector machines
- Ensemble methods

Summary: Linear Regression

Key Concepts

- Model: $\hat{y} = \mathbf{w}^T \mathbf{x}$
- Loss: Mean Squared Error
- Optimization: Normal equation or gradient descent
- Regularization: Ridge (L2) and Lasso (L1)
- Evaluation: MSE, RMSE, MAE, R^2

Extensions:

- Polynomial regression for non-linearity
- Robust regression (e.g., Huber loss)
- Bayesian linear regression
- Generalized linear models (GLMs)

Summary: Logistic Regression

Key Concepts

- Model: $P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$
- Loss: Binary cross-entropy
- Optimization: Gradient descent (no closed-form)
- Decision boundary: Linear in input space
- Evaluation: Accuracy, precision, recall, F1, AUC-ROC

Extensions:

- Multi-class: Softmax regression
- Kernel logistic regression
- Bayesian logistic regression

Comparison: Linear vs Logistic Regression

Aspect	Linear Regression	Logistic Regression
Task	Regression	Classification
Output	Continuous ($\mathbb{R}$)	Probability ($[0, 1]$)
Model	$\mathbf{w}^T \mathbf{x}$	$\sigma(\mathbf{w}^T \mathbf{x})$
Loss	MSE	Cross-entropy
Optimization	Closed-form/GD	Gradient descent
Assumptions	Linear relationship	Linear decision boundary
Evaluation	R^2 , RMSE	Accuracy, AUC

Common: Both are linear in parameters, use gradient descent, need feature scaling

Connection to Neural Networks

Linear models are the foundation of neural networks!

Single neuron $\approx$ logistic regression:

$$\text{output} = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

Multi-layer perceptron: Stack linear transformations + activations

$$\mathbf{h}^{(l+1)} = \sigma(\mathbf{W}^{(l)} \mathbf{h}^{(l)} + \mathbf{b}^{(l)})$$

Key additions in neural networks:

- Multiple layers (depth)
- Non-linear activation functions
- Backpropagation for training
- More complex architectures (CNN, RNN, Transformers)

Thank you!

Any questions about linear models?